

Production Planning & Scheduling System

How a modular Excel/VBA engine translated current WIP, part-specific routing, throughput objectives, and work-center capacity into one operation-level weekly schedule.

OPERATING QUESTION

What should happen next, when can it physically happen, and when should throughput be recognized?

ARCHITECTURAL ANSWER

Separate route interpretation, business preference, physical feasibility, and reporting - then make one master schedule authoritative.

SYSTEM ARCHITECTURE & DATA LINEAGE

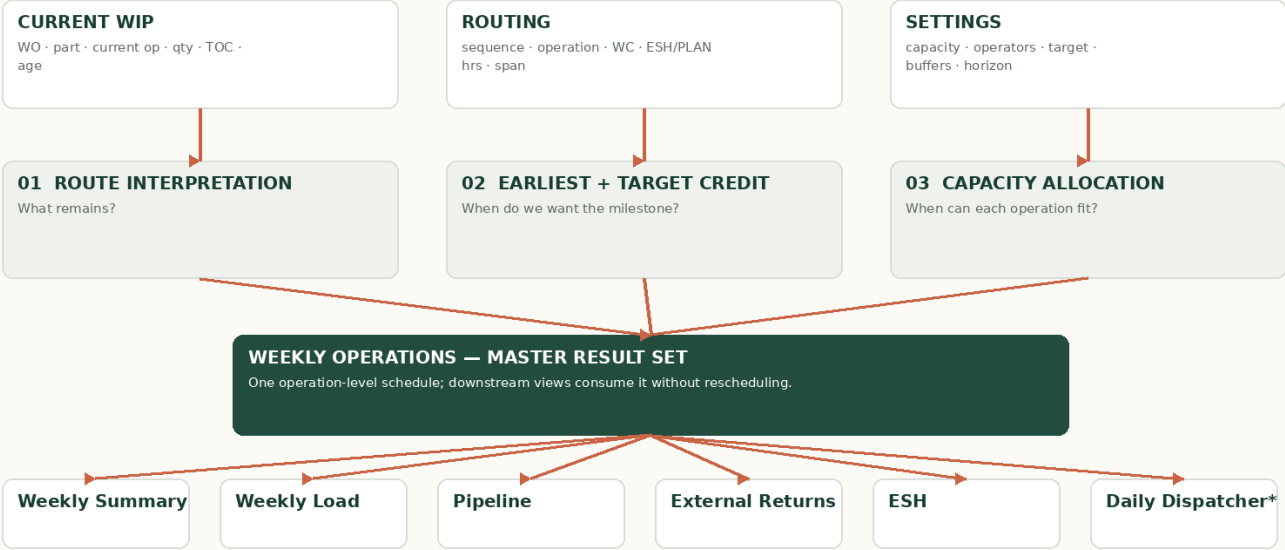


Figure 1. Source data, decision engines, master result set, and downstream consumers.

KEY PRINCIPLE

Raw data describes what exists. Routing describes what remains. Leveling chooses the preferred throughput date. Capacity determines what can physically fit. Weekly Operations records the final plan; reporting modules summarize it without rescheduling.

The workbook evolved away from a monolithic design because independent calculations across reports create conflicting schedules, double counting, and logic drift. The modular build treats sheet names, routing sources, and module dependencies as technical interfaces rather than incidental spreadsheet structure.

A preferred date is not the same thing as a feasible date.

The weekly engine first establishes the earliest date a work order could reach its designated throughput milestone. Only then can leveling choose a preferred credit date. Capacity allocation subsequently determines the actual day of each operation and, ultimately, the actual milestone date.

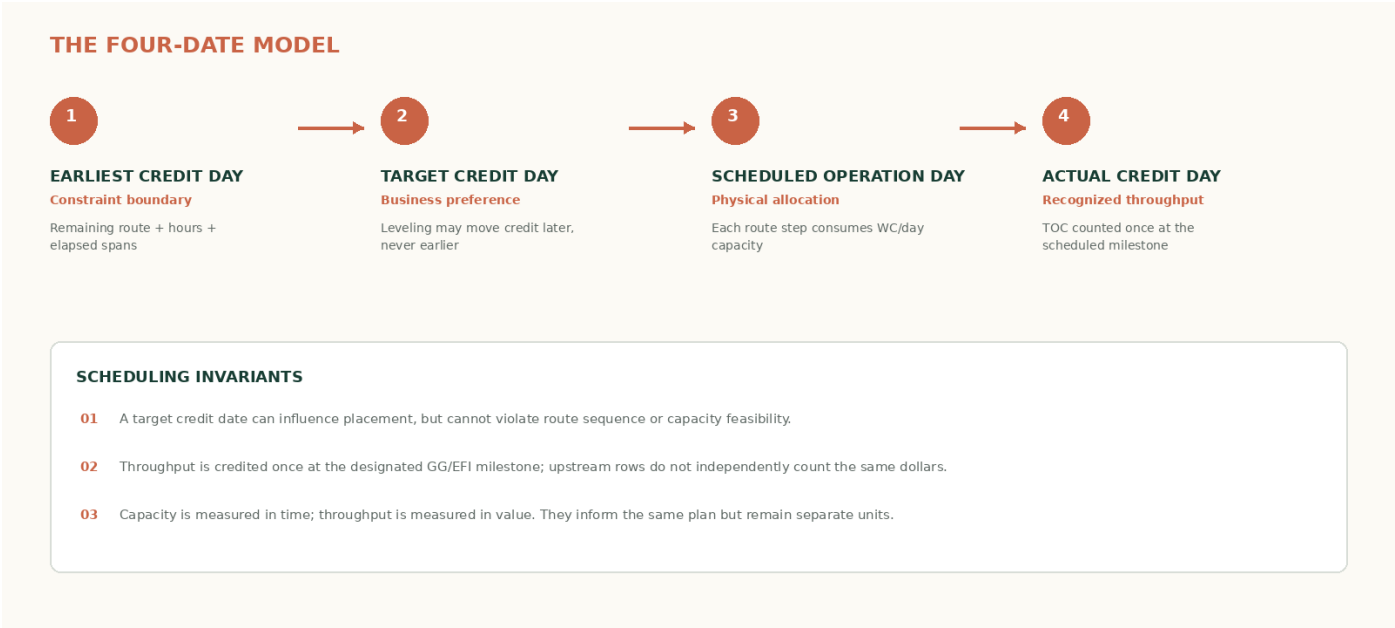


Figure 2. The four dates remain distinct so business objectives cannot silently override route or capacity constraints.

THROUGHPUT LEVELING

The scheduler seeks a balanced GG/EFI throughput profile rather than maximizing the earliest day. The documented operating intent was approximately \$280k-\$310k per day, subject to feasible flow.

CAPACITY ALLOCATION

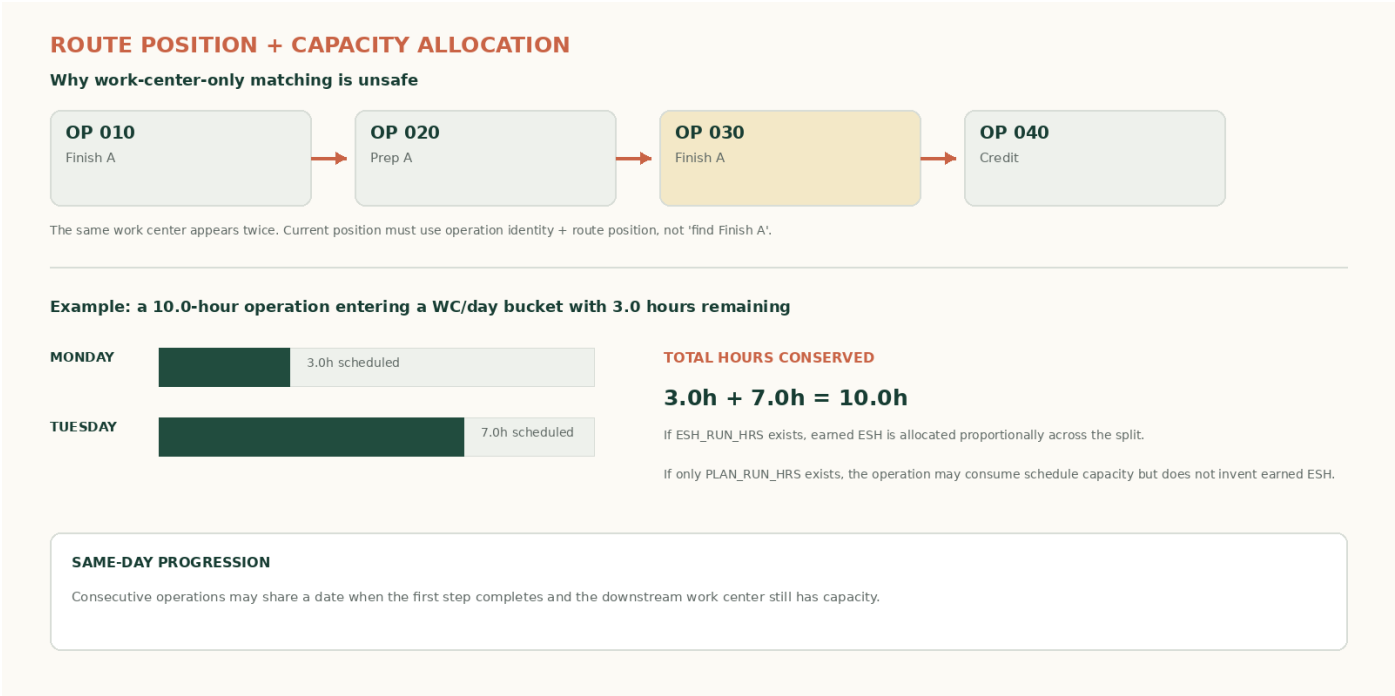
Capacity is shared across work orders in WC/day buckets. A job consumes only the hours that remain; excess work carries forward rather than overloading the day.

Weekly scheduling sequence

01-02	Read current WIP; match each WO to the correct point in its part-specific route.
03-04	Locate the next qualifying credit point; calculate earliest feasible arrival using remaining hours and elapsed spans.
05-06	Choose a target credit day; allocate remaining route operations against WC/day capacity.
07-08	Split operations when needed; permit same-day progression only when sequence and downstream capacity allow.
09-10	Write operation allocations to Weekly Operations; derive summary, load, pipeline, returns, and ESH from that result.

Route identity matters more than a work-center name.

A work center is a capacity pool, not a unique route position. Parts can revisit the same work center later in the route, so the scheduler must distinguish operation number, work center, and route position rather than treating those fields as interchangeable.



The scheduler's real complexity lives in the edge cases.

The engine does not force every work order through a generic fixed path. It preserves elapsed-time behavior, route identity, capacity constraints, and explicit exceptions so unusual work stays visible instead of being silently normalized.

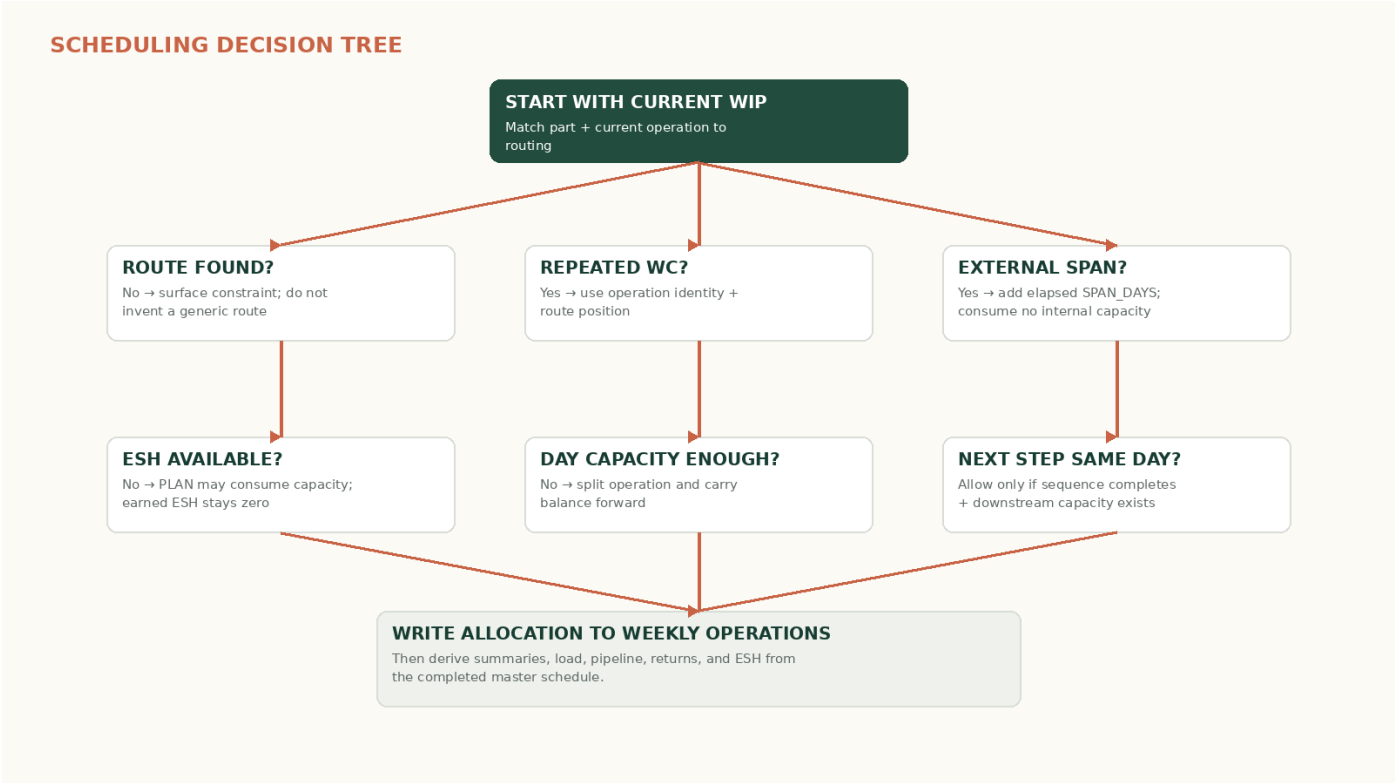


Figure 4. Decision tree for route, span, duration, split, and same-day progression cases.

External processing behaves like time, not labor

SPAN_DAYS

Represents elapsed time such as an external process. It shifts the earliest-return boundary but does not consume internal work-center capacity.

EXTERNAL RETURNS

The pipeline view uses scheduled external spans to estimate when work should re-enter internal flow and flags work that is late relative to that expected return.

Failure behavior is intentional

- Missing route: surface the work order as a constraint rather than inventing a generic route.
- No available capacity: push work forward within the horizon or show the constraint; target credit may differ from actual credit.
- Repeated work center: use operation identity and route position rather than a simple work-center lookup.
- Already near/at the milestone: schedule only remaining eligible work and avoid re-crediting completed stages.

Every downstream view must reconcile to the same master schedule.

Weekly Operations is the only operation-level master result set. Management and execution views may aggregate, filter, or reinterpret scheduled rows, but they should not independently decide when a work order belongs on a day.

VIEW	QUESTION ANSWERED	SOURCE / BOUNDARY
Weekly Summary	What throughput and plan are expected?	Summarizes Weekly Operations.
Weekly Load	Where is WC/day capacity loaded or constrained?	Consumes operation allocations + capacity results.
Pipeline Forecast	What work is expected to arrive into each WC?	Forward-flow view from scheduled routes.
External Returns	When should externally processed work return?	Uses scheduled spans / elapsed-time logic.
Daily ESH / Detail	What earned-hours profile does the plan imply?	Measures scheduled work; does not place it.
Daily Dispatcher	What should the floor work today?	Separate daily priority engine using current WIP + routing.

Post-run control checks

- No WO receives throughput credit more than once for the same event, and credit occurs at the intended route milestone.
- No WC/day exceeds configured capacity unless the constraint is explicitly surfaced.
- External returns are plausible relative to SPAN_DAYS; split operations conserve total hours and proportional ESH.
- Daily throughput is reasonably leveled unless route/capacity constraints make that impossible.
- Weekly Summary, Weekly Load, Pipeline, External Returns, and ESH reconcile back to Weekly Operations.

REGRESSION SET	Substantial logic changes should retain known test cases including at least one external span, one split operation, one repeated work center, and one zero-ESH fallback case.
----------------	---

Modularity protects the scheduling logic from the workbook around it.

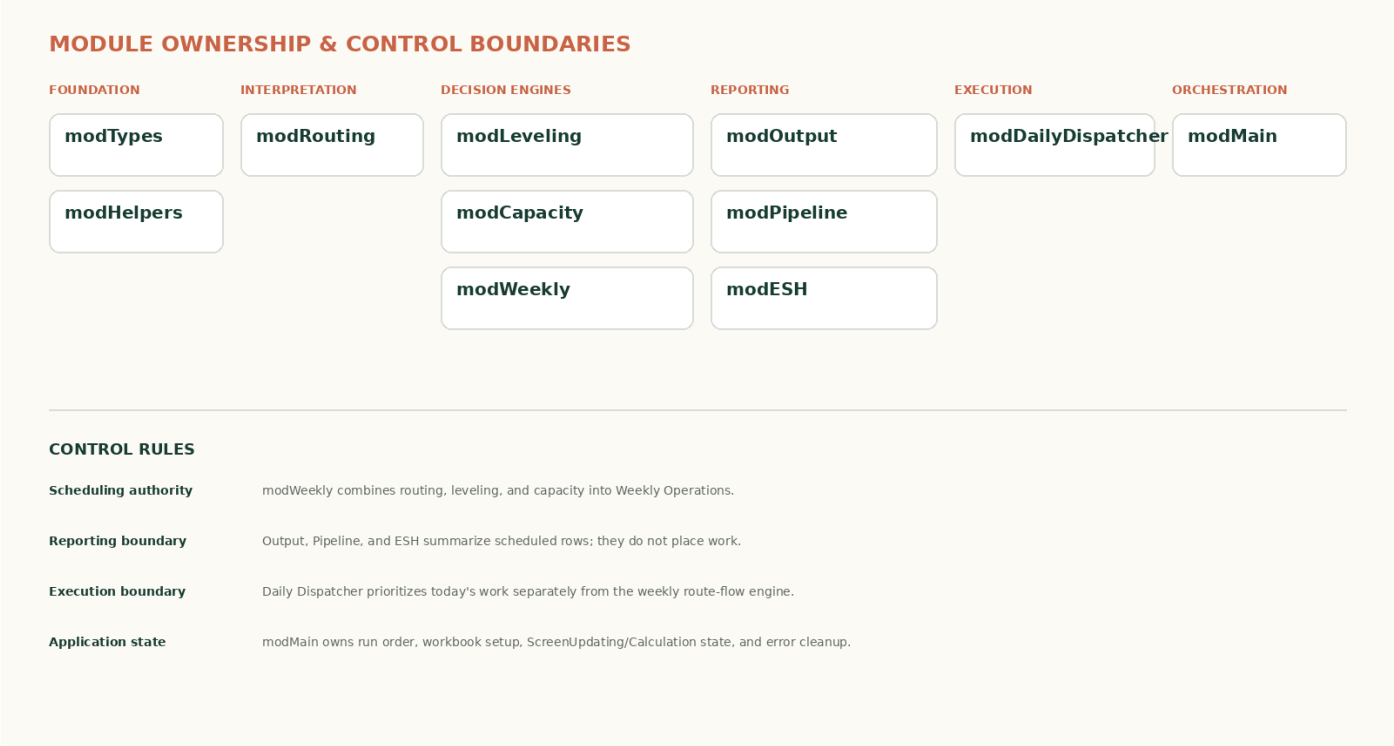


Figure 5. Module ownership separates foundational utilities, route interpretation, decision engines, reporting, daily execution, and orchestration.

IMPLEMENTATION PATTERN

Shared types, arrays, dictionaries, and collections are preferred over cell-by-cell inner loops. modMain owns workbook/application state and restores Excel after failures.

INTERFACE DISCIPLINE

Routing-source names, expected sheet names, and module import order function like API contracts. Output-formatting failures should be recoverable without changing the schedule.

Human review remains part of the operating model

The practical run sequence is intentionally short: refresh the WIP snapshot, confirm routing and settings, run the engine, review Weekly Operations first, inspect load and throughput balance, review pipeline/returns and ESH, make limited WIP swaps when real-world context requires them, then publish the plan.

OBSERVED
IMPACT

Routine planning work dropped from approximately 13-19 hours per week to roughly 5-10 minutes per day under normal conditions, while making aged WIP, late returns, expected throughput, and missed assignments easier to see.

Next iteration

- Connect attendance and labor availability more directly to capacity assumptions.
- Integrate workforce capability data so staffing feasibility can reflect who is qualified for each work center.
- Add automated alerts for late returns, route gaps, overloads, and missed assignments.
- Reduce manual refreshes through more direct source-system inputs.

Portfolio note: reconstructed from the original August 20, 2026 SOP and technical architecture using generalized terminology and synthetic records. Proprietary production data and facility-specific identifiers are excluded.